

[Name der Hochschule]

[Art der Arbeit]

[Titel der Fallstudie zum Technologie-Template]

[Untertitel]

[Aufgabenstellung]

Kurs: [Kurs / Modul]
Studiengang: [Studiengang]
Autor: [Vorname Nachname]
Matrikelnummer: [Matrikelnummer]
Tutor: [Tutor / Betreuer]
Abgabedatum: [Monat Jahr]

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Ausgangslage	1
1.2 Problemstellung	1
1.3 Zielsetzung	2
1.4 Fragestellungen	2
1.5 Methodisches Vorgehen	3
1.6 Aufbau der Fallstudie	4
2 Anforderungen an das Technologie-Template	5
2.1 Zielbild des Technologie-Templates	5
2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung	5
2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen	5
2.4 Qualitäts- und Wartbarkeitsanforderungen	5
2.5 Abgrenzung	5
3 Architektur des Technologie-Templates	5
3.1 Gesamtarchitektur	5
3.2 Frontend mit Vite und TypeScript	6
3.3 Backend mit FastAPI	7
3.4 Desktop-Integration mit Tauri und Rust	7
3.5 Shared Contracts und Schnittstellen	7
3.6 Konfigurationsmodell	7
3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic	8
4 Projektprofile und Feature-Modell	8
4.1 Grundprinzip des Profilmodells	8
4.2 Profil Web-only	10
4.3 Profil Web-cloud	10

4.4	Profil Desktop-local	10
4.5	Profil Desktop-cloud	10
4.6	Profil Full-platform	10
4.7	Optionale Capabilities	10
4.8	Single-Repository-Strategie	10
5	Tooling und Entwicklungsworkflow	10
5.1	Rolle von control.py	10
5.2	Projektinitialisierung und Generierung	11
5.3	Systemprüfung und Installation	11
5.4	Entwicklungsbetrieb	11
5.5	Tests und Builds	12
5.6	Release- und Packaging-Workflow	12
5.7	Entwickler-Onboarding	12
6	Qualitätssicherung und Verifikation	12
6.1	Teststrategie	12
6.2	Continuous Integration	13
6.3	Abnahme und technische Verifikation	13
6.4	Reproduzierbarkeit	13
6.5	Security und Konfigurationsgrenzen	13
6.6	Extern verbleibende Prüfungen	13
7	Bewertung des Technologie-Templates	13
7.1	Produktivitätswirkung	13
7.2	Stärken	13
7.3	Schwächen und Grenzen	14
7.4	Wartungsaufwand	14
7.5	Vergleich mit alternativen Template-Strategien	14
7.6	Gesamtbewertung	14
8	Einsatzmöglichkeiten und Transfer auf Kundenprojekte	14
8.1	Geeignete Projekttypen	14
8.2	Ungeeignete und eingeschränkt geeignete Projekttypen	14
8.3	Analyse von Kundenanforderungen	14

8.4	Auswahl des Projektprofils	14
8.5	Generierung eines Kundenprojekts	14
8.6	Überführung in die Produktentwicklung	15
9	Schlussbetrachtung	15
9.1	Zusammenfassung der Ergebnisse	15
9.2	Beantwortung der Fragestellungen	15
9.3	Kritische Würdigung	15
9.4	Ausblick	15
9.5	Fazit	15
	Literaturverzeichnis	16

Abbildungsverzeichnis

1	Methodische Schritte der Fallstudie	4
2	Gesamtarchitektur des Technologie-Templates	5
3	Komponenten und Verantwortungsgrenzen im Master Repository	5
4	Profilabhängige Laufzeitarchitektur	6
5	Konfigurationspriorität und Sicherheitsgrenzen	7
6	Profilmodell auf Basis eines gemeinsamen Master-Templates	8
7	Strukturelle Feature-Zuordnung der Projektprofile	8
8	Entscheidungsstruktur für die Auswahl eines Projektprofils	9
9	Befehlsgruppen des zentralen Projekt-Toolings	10
10	Workflow der profilgesteuerten Projektgenerierung	11
11	Grundstruktur des Entwicklungsworkflows	11
12	Providerneutrale Deployment-Architektur	12
13	Plattformübergreifendes Modell für Desktop-Releases	12
14	Nachweiskette der technischen Verifikation	12
15	Struktur der profilbezogenen Verifikationsmatrix	13
16	Offene Bewertungsstruktur für die Projekteignung	14
17	Prozess vom Kundenbedarf bis zur Auslieferung	14

Tabellenverzeichnis

1	Struktur zur Dokumentation des Technologie-Stacks	6
2	Struktur zur Dokumentation der Projektprofile	9
3	Struktur zur Dokumentation der technischen Verifikation	13
4	Struktur zur Gegenüberstellung von Stärken und Grenzen	13
5	Struktur zur Bewertung der Projekteignung	14

Abkürzungsverzeichnis

Abk.	Bedeutung
------	-----------

1 Einleitung

1.1 Ausgangslage

Die Einrichtung eines neuen Softwareprojekts umfasst neben der Umsetzung der Fachlogik zahlreiche technische Grundentscheidungen. Dazu gehören die Auswahl und Konfiguration der verwendeten Technologien, die Strukturierung des Quellcodes, die Definition von Schnittstellen sowie die Einrichtung von Entwicklungs-, Test-, Build- und Deployment-Prozessen. Bei Projekten mit vergleichbaren technischen Anforderungen treten diese Aufgaben wiederholt auf. Werden sie für jedes Vorhaben unabhängig gelöst, entstehen bereits vor Beginn der eigentlichen Produktentwicklung unterschiedliche technische Ausgangslagen.

Die Anforderungen an diese Ausgangslagen unterscheiden sich zusätzlich nach der vorgesehenen Betriebs- und Nutzungsform. Eine reine Web-Anwendung benötigt andere Komponenten als eine cloudgestützte Anwendung mit Backend und Persistenz. Desktop-Anwendungen erfordern darüber hinaus eine Integration in das jeweilige Betriebssystem sowie eigene Build- und Bereitstellungsmechanismen. Soll ein Projekt mehrere Plattformen unterstützen, müssen gemeinsame Bestandteile und plattformspezifische Erweiterungen so voneinander abgegrenzt werden, dass sie konsistent entwickelt und gewartet werden können.

Vor diesem Hintergrund besteht Bedarf an einer standardisierten technischen Ausgangsbasis, die wiederkehrende Entscheidungen nachvollziehbar bündelt und zugleich unterschiedliche Projektformen berücksichtigt. Ein konsistentes Entwicklungsmodell kann hierfür gemeinsame Strukturen, Werkzeuge und Qualitätssicherungsmaßnahmen bereitstellen. Gegenstand dieser Fallstudie ist das Projekt „Template-Projekte“, das als wiederverwendbares Full-Stack-Technologie-Template für Web-, Cloud- und Desktop-Projekte konzipiert ist. Seine konkrete Ausgestaltung und seine Eignung werden im weiteren Verlauf der Arbeit systematisch untersucht (kleiveist, 2026d).

1.2 Problemstellung

Die wiederholte Initialisierung technisch ähnlicher Softwareprojekte bindet Aufwand, der nicht unmittelbar der Umsetzung produktspezifischer Anforderungen dient. Projektstrukturen, Konfigurationen und Automatisierungen müssen erneut erstellt, aufeinander abgestimmt und überprüft werden. Ohne eine gemeinsame Baseline können sich dabei selbst zwischen verwandten Projekten Unterschiede in Architektur, Werkzeugauswahl, Schnittstellengestaltung und Entwicklungsabläufen ergeben. Diese Unterschiede erschweren die Übertragung von Erfahrungen und erhöhen den Einarbeitungsbedarf bei einem Wechsel zwischen Projekten.

Werden für Web-, Cloud- und Desktop-Anwendungen getrennte technische Ausgangsstände gepflegt, entsteht zusätzlicher Wartungsaufwand. Änderungen an gemeinsam verwendeten Technologien oder Qualitätsanforderungen müssen dann in mehreren Baselines nachvollzogen werden. Erfolgt diese Pflege nicht gleichförmig, können Technologieverversionen, Build-Prozesse, Konfigurationsmodelle und Dokumentationsstände auseinanderlaufen. Damit steigt das Risiko, dass vergleichbare Anforderungen unterschiedlich umgesetzt oder Prüfungen nur in einzelnen Projektvarianten berücksichtigt werden.

Die zentrale Herausforderung besteht folglich darin, Einheitlichkeit und Flexibilität miteinander zu verbinden. Ein Technologie-Template muss genügend gemeinsame Struktur vorgeben, um Standardisierung

und Wiederverwendung zu ermöglichen. Gleichzeitig darf es Projekte nicht mit Komponenten oder Abhängigkeiten belasten, die für das gewählte Einsatzszenario nicht benötigt werden. Die Fallstudie untersucht deshalb, wie das entwickelte Template diesen Zielkonflikt durch eine gemeinsame technische Basis, Projektprofile und optionale Erweiterungen adressiert und an welchen Stellen Grenzen oder zusätzlicher Anpassungsbedarf bestehen (kleiveist, 2026a, 2026b).

1.3 Zielsetzung

Ziel der Fallstudie ist eine strukturierte Untersuchung des Projekts „Template-Projekte“. Betrachtet wird das darin entwickelte Technologie-Template. Hierzu werden seine Architektur, die Aufteilung in gemeinsam genutzte und profilspezifische Bestandteile sowie die unterstützenden Entwicklungs- und Qualitätssicherungsprozesse analysiert. Die Betrachtung richtet sich damit nicht nur auf die eingesetzten Technologien, sondern insbesondere auf deren Zusammenwirken innerhalb einer wiederverwendbaren technischen Ausgangsbasis (kleiveist, 2026a, 2026e).

Auf Grundlage dieser Analyse soll bewertet werden, in welchem Umfang das Template Wiederverwendung und Standardisierung unterstützt und welches Potenzial sich daraus für die Produktivität bei der Initialisierung und Weiterentwicklung von Softwareprojekten ergibt. Darüber hinaus wird untersucht, wie die vorgesehenen Projektprofile unterschiedliche Anforderungen von Web-, Cloud- und Desktop-Anwendungen abbilden. Die Bewertung erfolgt anhand der im Projekt vorliegenden Strukturen, Implementierungen und Verifikationsnachweise; quantitative Effekte werden nur berücksichtigt, sofern hierfür belastbare Daten verfügbar sind (kleiveist, 2026c).

Als anwendungsbezogenes Ergebnis sollen geeignete Einsatzfelder und relevante Auswahlkriterien für zukünftige Kundenprojekte abgeleitet werden. Zugleich sind technische Grenzen, externe Voraussetzungen und projektspezifisch verbleibende Aufgaben herauszuarbeiten. Nicht Ziel der Untersuchung ist es, das Template als universelle Lösung für sämtliche Softwareklassen einzuordnen oder eine produktspezifische Facharchitektur vorzugeben.

1.4 Fragestellungen

Aus der beschriebenen Problemstellung und Zielsetzung ergeben sich sechs zentrale Untersuchungsfragen. Sie strukturieren die Analyse des Templates und bilden den Bezugsrahmen für seine spätere Bewertung:

1. Welche fachlich-technischen, qualitativen und betrieblichen Anforderungen werden durch das Technologie-Template adressiert?
2. Wie unterstützen die Architektur und das Profilmodell die Bereitstellung unterschiedlicher Varianten für Web-, Cloud- und Desktop-Projekte?
3. Inwiefern fördern der gemeinsame technische Kern und die Single-Repository-Strategie die Wiederverwendung und Standardisierung zwischen Projekten?
4. Welches Potenzial bietet das Template, wiederkehrende Aufwände bei Projektinitialisierung, Entwicklungsworkflow, Qualitätssicherung und Bereitstellung zu reduzieren?

5. Für welche Projektarten und Kundenanforderungen stellt das Template eine geeignete technische Ausgangsbasis dar?
6. Welche technischen Grenzen, Wartungsaufwände, Risiken und externen Abhängigkeiten sind bei seinem Einsatz zu berücksichtigen?

Die Fragen sind miteinander verknüpft: Aussagen zu Produktivität und Eignung setzen zunächst eine nachvollziehbare Betrachtung der Anforderungen, der Architektur und der vorhandenen Verifikation voraus. Ihre Beantwortung wird daher nicht isoliert, sondern entlang der aufeinander aufbauenden Kapitel der Fallstudie vorgenommen.

1.5 Methodisches Vorgehen

Die Untersuchung ist als technische Fallstudie des implementierten Templates angelegt (Runeson & Höst, 2009). Ausgangspunkt ist eine Bestandsaufnahme des Master Repository. Dabei werden die Repository-Struktur, die enthaltenen Komponenten und die dokumentierten Verantwortungsgrenzen erfasst. Anschließend werden die Architekturentscheidungen und die Beziehungen zwischen Frontend, Backend, Desktop-Integration, gemeinsam genutzten Schnittstellen, Persistenz und Deployment betrachtet. Die Analyse beschreibt den vorhandenen Gegenstand und trennt belegbare Projekteigenschaften von Annahmen oder noch offenen Prüfpunkten (kleiveist, 2026a).

Darauf aufbauend wird das Profilmodell untersucht. Im Mittelpunkt stehen die Abgrenzung der Profile, die Aufteilung zwischen gemeinsamem Kern und profilspezifischen Bestandteilen sowie das Modell optionaler Capabilities. Die Betrachtung des Python-basierten Toolings ergänzt diese strukturelle Analyse um den praktischen Entwicklungsablauf. Hierzu werden insbesondere Projektgenerierung, Systemprüfung, Installation, Entwicklungsbetrieb, Tests, Builds und Release-Vorbereitung als zusammenhängender Workflow betrachtet. Die maßgeblichen Profil- und Werkzeugbeschreibungen werden dabei als projektinterne Primärquellen herangezogen (kleiveist, 2026b, 2026e).

Für die Verifikation werden vorhandene automatisierte Tests, Build-Ergebnisse und dokumentierte Abnahmen den jeweils untersuchten Anforderungen und Architekturentscheidungen zugeordnet. Externe oder produktspezifische Prüfschritte werden davon abgegrenzt. Auf dieser Evidenzbasis erfolgt die Bewertung von Wiederverwendbarkeit, Standardisierung, Produktivitätspotenzial, Stärken und Grenzen. Abschließend werden daraus Kriterien für geeignete Einsatzfelder und für die Übertragung auf zukünftige Kundenprojekte abgeleitet (kleiveist, 2026c).

Abbildung 1 fasst diese aufeinander aufbauenden Untersuchungsschritte zusammen. Die vertikale Darstellung verdeutlicht, dass die Bewertung und der Transfer erst auf Grundlage der zuvor erhobenen und analysierten Projektinformationen erfolgen.

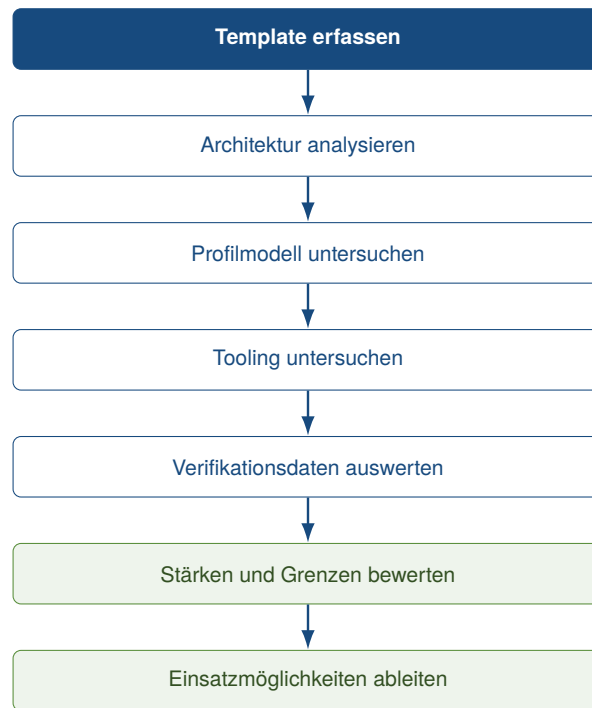


Abbildung 1: Methodische Schritte der Fallstudie

1.6 Aufbau der Fallstudie

Nach der Einleitung konkretisiert Abschnitt 2 die Anforderungen und die Abgrenzung des Technologie-Templates. Darauf folgt in Abschnitt 3 die Untersuchung seiner Gesamtarchitektur und der beteiligten technischen Komponenten. Abschnitt 4 betrachtet anschließend das Profil- und Feature-Modell einschließlich der optionalen Capabilities und der Single-Repository-Strategie. Das zentrale Projekt-Tooling sowie der Entwicklungs-, Build- und Release-Workflow werden in Abschnitt 5 untersucht.

Abschnitt 6 behandelt die Qualitätssicherung, die technische Verifikation und extern verbleibende Prüfungen. Auf dieser Grundlage bewertet Abschnitt 7 das Template hinsichtlich Produktivitätswirkung, Stärken, Grenzen und Wartungsaufwand. Abschnitt 8 überträgt die gewonnenen Erkenntnisse auf die Auswahl und Generierung zukünftiger Kundenprojekte. Die Arbeit endet mit der Schlussbetrachtung in Abschnitt 9, in der die Ergebnisse gebündelt, die Fragestellungen beantwortet und offene Entwicklungsperspektiven eingeordnet werden.

2 Anforderungen an das Technologie-Template

2.1 Zielbild des Technologie-Templates

2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung

2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen

2.4 Qualitäts- und Wartbarkeitsanforderungen

2.5 Abgrenzung

3 Architektur des Technologie-Templates

3.1 Gesamtarchitektur

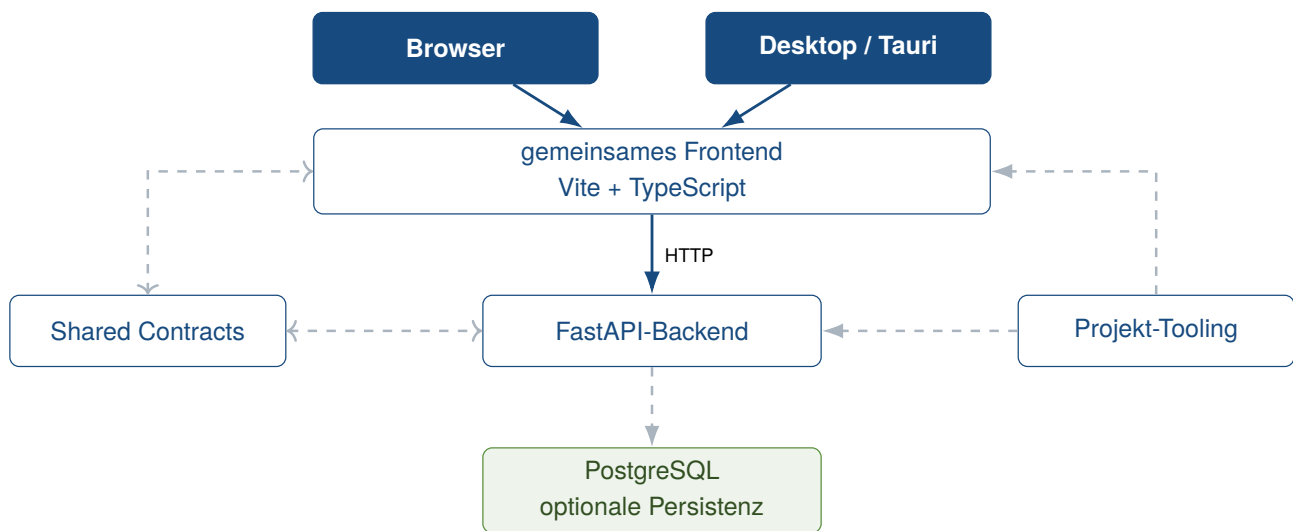


Abbildung 2: Gesamtarchitektur des Technologie-Templates

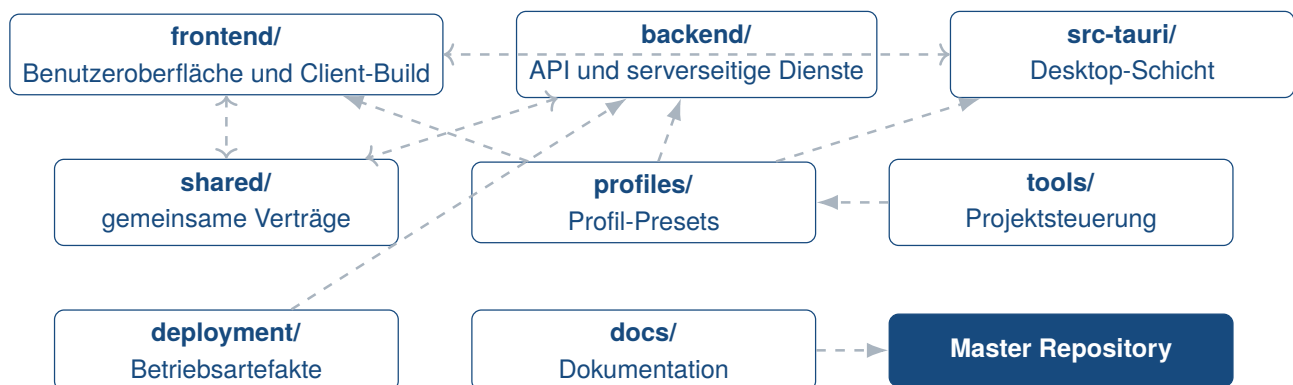
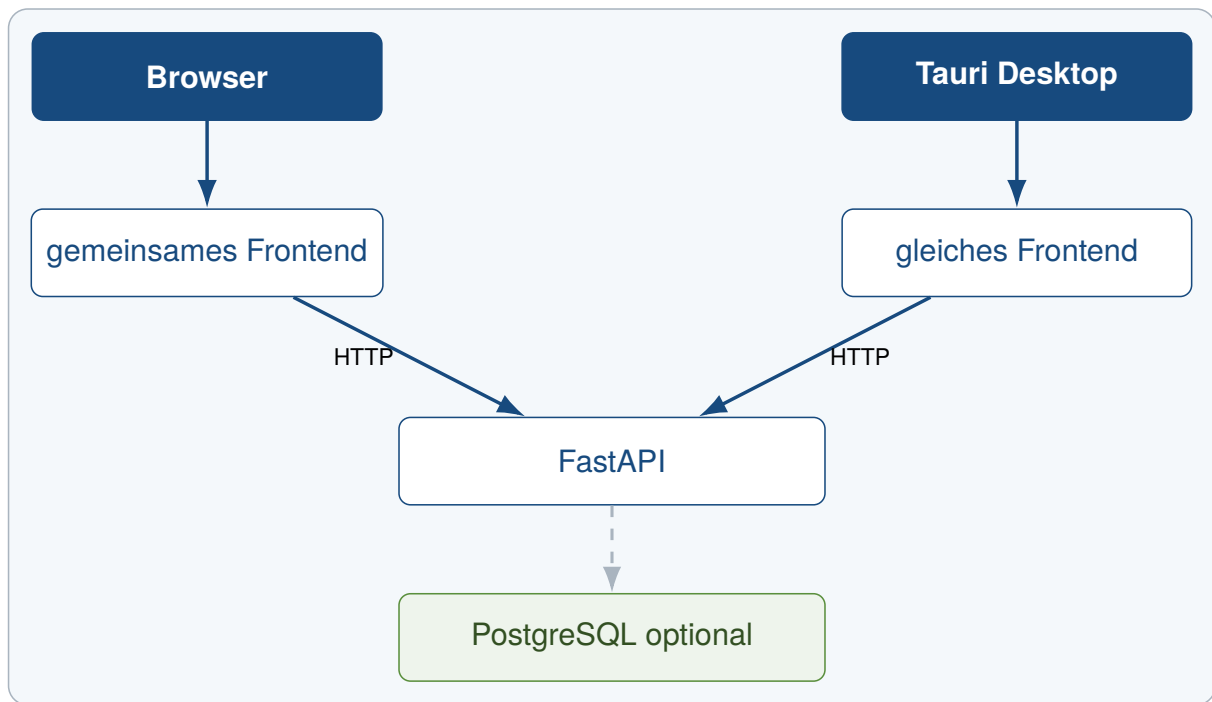


Abbildung 3: Komponenten und Verantwortungsgrenzen im Master Repository



Komponentenbestand abhängig vom gewählten Profil

Abbildung 4: Profilabhängige Laufzeitarchitektur

3.2 Frontend mit Vite und TypeScript

Tabelle 1: Struktur zur Dokumentation des Technologie-Stacks

Technologie	Rolle im Template	Version / Quelle	Projektbeleg
-------------	-------------------	------------------	--------------

3.3 Backend mit FastAPI

3.4 Desktop-Integration mit Tauri und Rust

3.5 Shared Contracts und Schnittstellen

3.6 Konfigurationsmodell

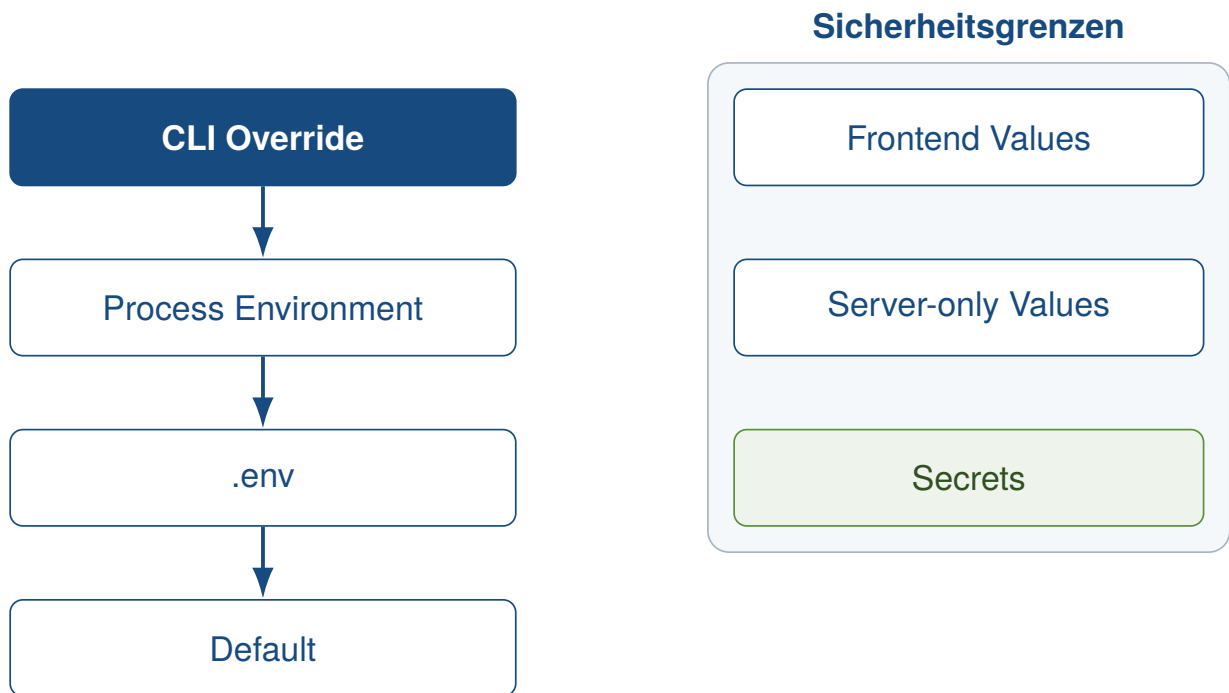


Abbildung 5: Konfigurationspriorität und Sicherheitsgrenzen

3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic

4 Projektprofile und Feature-Modell

4.1 Grundprinzip des Profilmodells

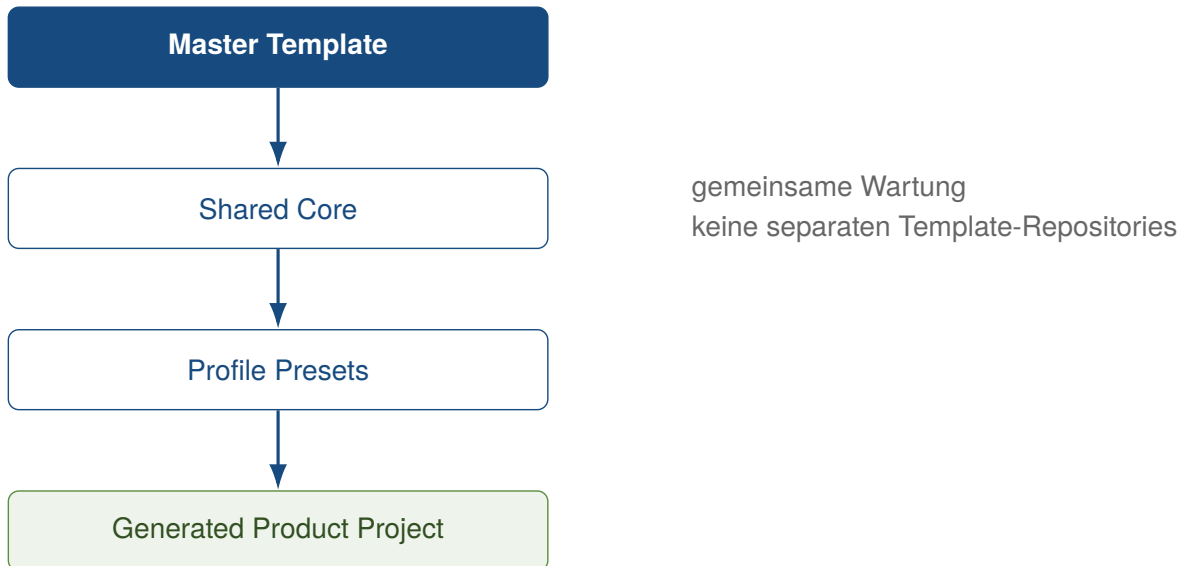


Abbildung 6: Profilmodell auf Basis eines gemeinsamen Master-Templates

Profil	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	enthalten	–	–	–	–
web-cloud	enthalten	enthalten	–	enthalten	optional
desktop-local	enthalten	–	enthalten	–	–
desktop-cloud	enthalten	enthalten	enthalten	enthalten	optional
full-platform	enthalten	enthalten	enthalten	enthalten	optional

Abbildung 7: Strukturelle Feature-Zuordnung der Projektprofile

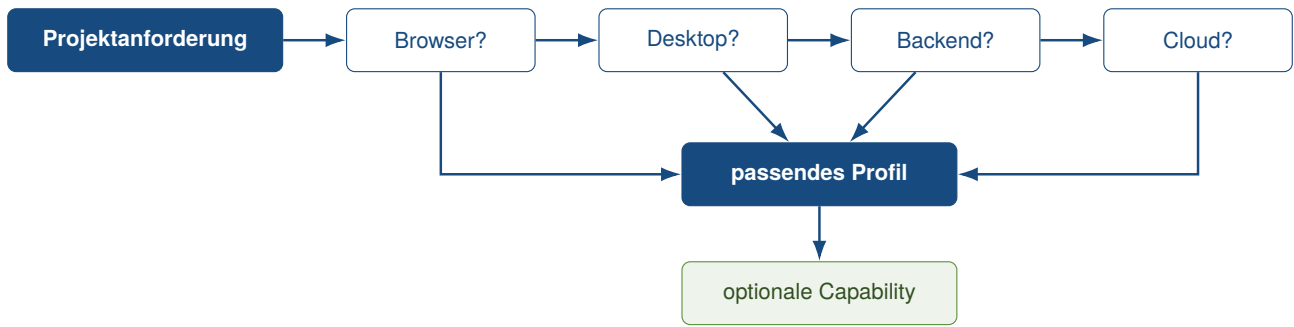


Abbildung 8: Entscheidungsstruktur für die Auswahl eines Projektprofils

Tabelle 2: Struktur zur Dokumentation der Projektprofile

Profil	Plattformbezug	enthaltene Komponenten	optionale Capabilities
--------	----------------	------------------------	------------------------

4.2 Profil Web-only

4.3 Profil Web-cloud

4.4 Profil Desktop-local

4.5 Profil Desktop-cloud

4.6 Profil Full-platform

4.7 Optionale Capabilities

4.8 Single-Repository-Strategie

5 Tooling und Entwicklungsworkflow

5.1 Rolle von control.py

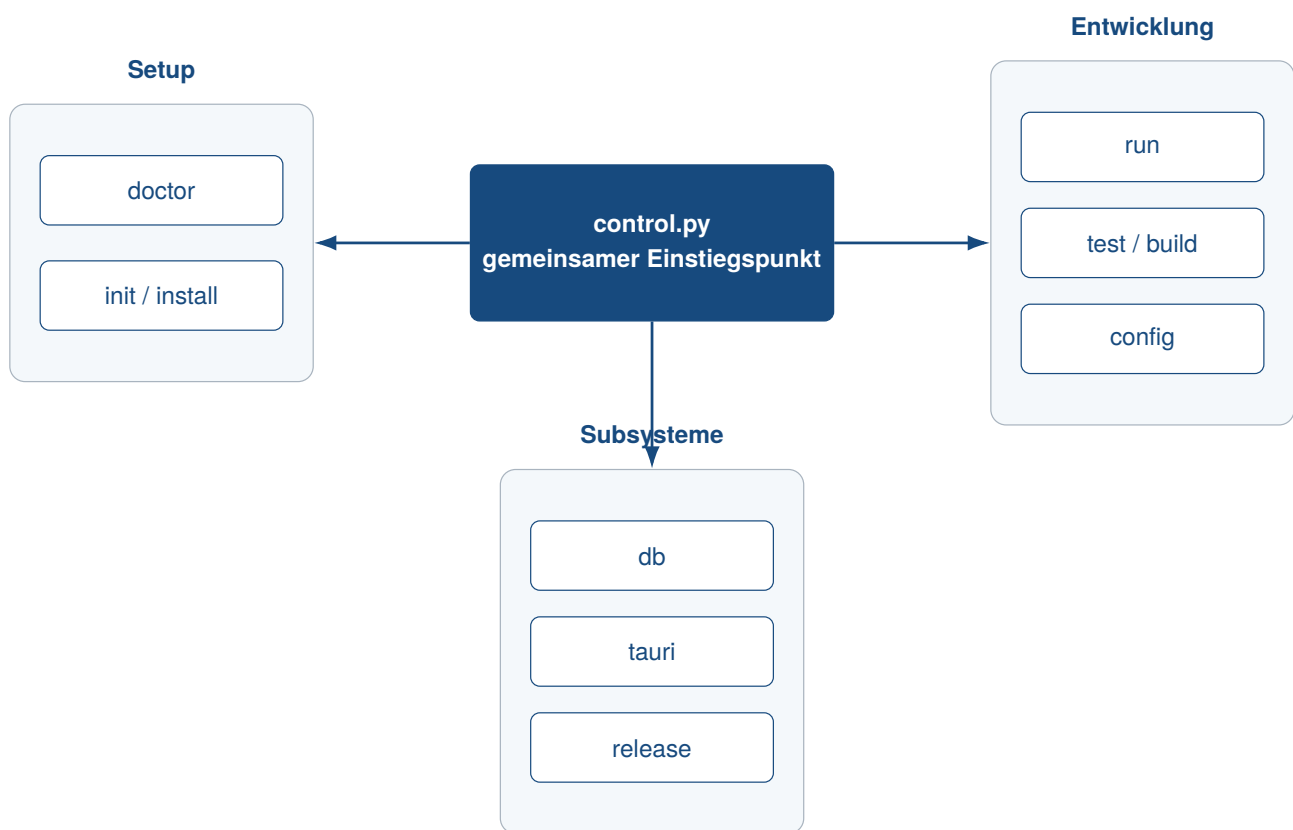


Abbildung 9: Befehlsgruppen des zentralen Projekt-Toolings

5.2 Projektinitialisierung und Generierung

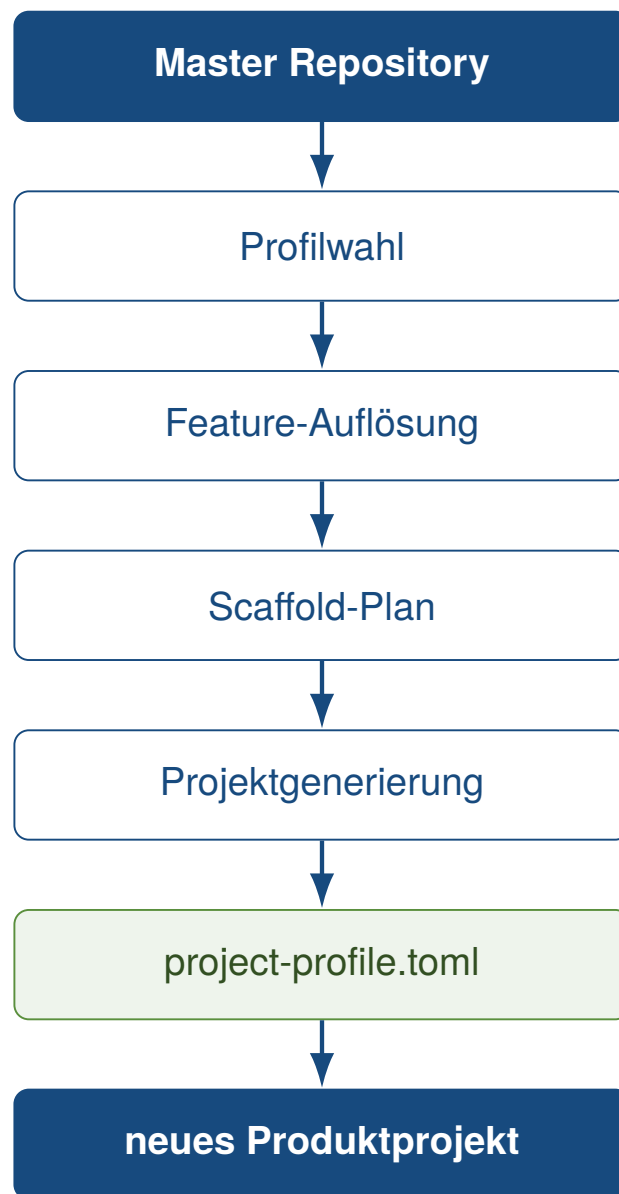


Abbildung 10: Workflow der profilgesteuerten Projektgenerierung

5.3 Systemprüfung und Installation

5.4 Entwicklungsbetrieb



Abbildung 11: Grundstruktur des Entwicklungsworkflows

5.5 Tests und Builds

5.6 Release- und Packaging-Workflow

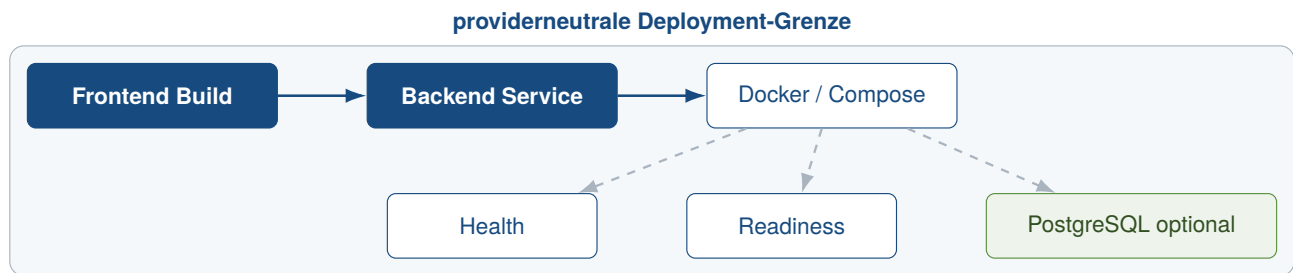


Abbildung 12: Providerneutrale Deployment-Architektur

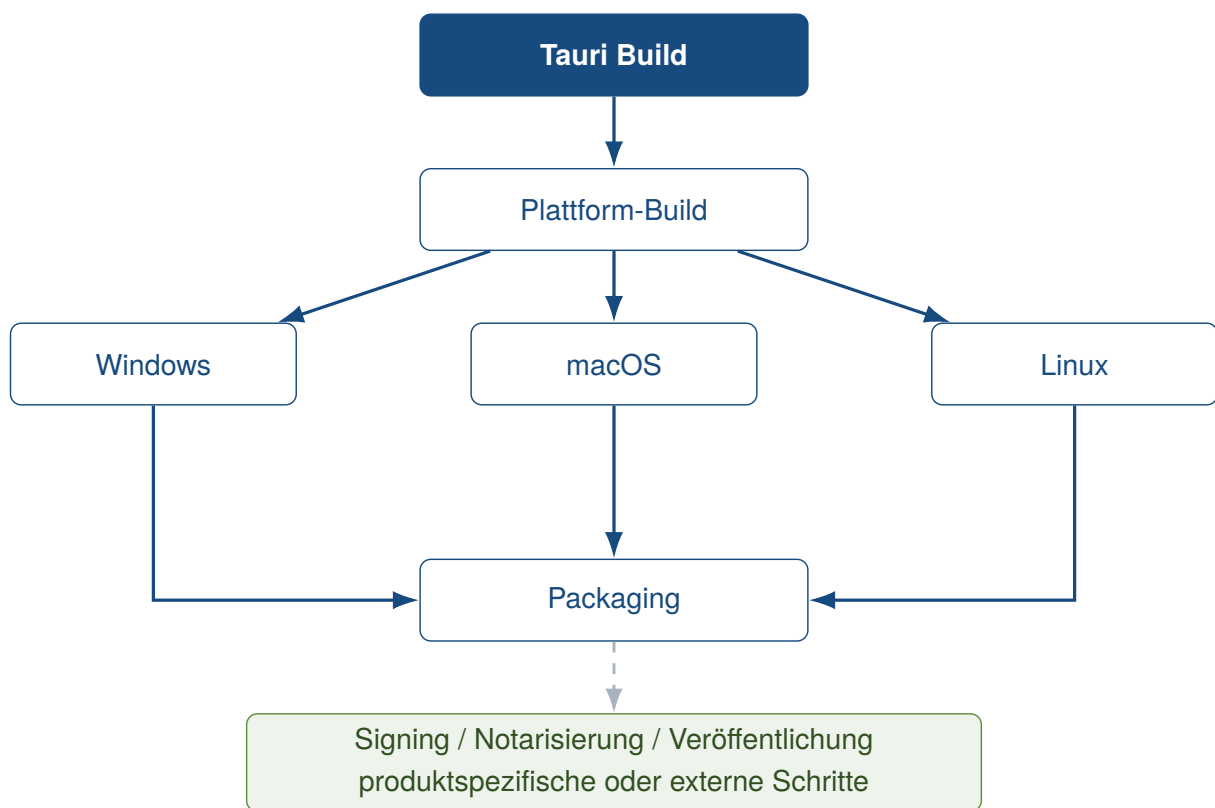


Abbildung 13: Plattformübergreifendes Modell für Desktop-Releases

5.7 Entwickler-Onboarding

6 Qualitätssicherung und Verifikation

6.1 Teststrategie



Abbildung 14: Nachweiskette der technischen Verifikation

6.2 Continuous Integration

6.3 Abnahme und technische Verifikation

Profil	Generate	Install	Doctor	Tests	Web Build	Desktop / Tauri	Database	Container
web-only	offen	offen	offen	offen	offen	offen	offen	offen
web-cloud	offen	offen	offen	offen	offen	offen	offen	offen
desktop-local	offen	offen	offen	offen	offen	offen	offen	offen
desktop-cloud	offen	offen	offen	offen	offen	offen	offen	offen
full-platform	offen	offen	offen	offen	offen	offen	offen	offen

Abbildung 15: Struktur der profilbezogenen Verifikationsmatrix

Tabelle 3: Struktur zur Dokumentation der technischen Verifikation

Profil	Prüfbereich	Kriterium	Evidenz	Ergebnis
--------	-------------	-----------	---------	----------

6.4 Reproduzierbarkeit

6.5 Security und Konfigurationsgrenzen

6.6 Extern verbleibende Prüfungen

7 Bewertung des Technologie-Templates

7.1 Produktivitätswirkung

7.2 Stärken

Tabelle 4: Struktur zur Gegenüberstellung von Stärken und Grenzen

Aspekt	Stärke / Evidenz	Grenze / Evidenz	Auswirkung
--------	------------------	------------------	------------

7.3 Schwächen und Grenzen

7.4 Wartungsaufwand

7.5 Vergleich mit alternativen Template-Strategien

7.6 Gesamtbewertung

8 Einsatzmöglichkeiten und Transfer auf Kundenprojekte

8.1 Geeignete Projekttypen

Projekttyp	Anforderungsfit	Anpassungsbedarf	Evidenz
Web-Anwendungen	offen	offen	offen
Cloud-Anwendungen	offen	offen	offen
Desktop-Anwendungen	offen	offen	offen
SaaS + Desktop	offen	offen	offen
lokale Business-Tools	offen	offen	offen
stark native Anwendungen	offen	offen	offen
Echtzeitsysteme	offen	offen	offen
Spiele	offen	offen	offen

Abbildung 16: Offene Bewertungsstruktur für die Projekteignung

Tabelle 5: Struktur zur Bewertung der Projekteignung

Projekttyp	Anforderungsfit	Anpassungsbedarf	Grenzen	Evidenz
------------	-----------------	------------------	---------	---------

8.2 Ungeeignete und eingeschränkt geeignete Projekttypen

8.3 Analyse von Kundenanforderungen

8.4 Auswahl des Projektprofils

8.5 Generierung eines Kundenprojekts



Abbildung 17: Prozess vom Kundenbedarf bis zur Auslieferung

8.6 Überführung in die Produktentwicklung

9 Schlussbetrachtung

9.1 Zusammenfassung der Ergebnisse

9.2 Beantwortung der Fragestellungen

9.3 Kritische Würdigung

9.4 Ausblick

9.5 Fazit

Literaturverzeichnis

- kleiveist. (2026a, 6. August). *Framework architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/architecture.md>
- kleiveist. (2026b, 6. August). *Project profiles* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/project-profiles.md>
- kleiveist. (2026c, 7. August). *Template final acceptance* [Technisches Abnahmeprotokoll im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md>
- kleiveist. (2026d, 7. August). *Template-projekte: Full-stack project template* [GitHub-Repository, geprüfter Commit a4487ac]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte>
- kleiveist. (2026e, 6. August). *Tooling guide* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/tooling.md>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>